2D 타워 디펜스 맵 설계의 모든 것: 유니티 타일 맵 마스터하기

기초 게임 로직부터 리소스 가공, 타일 팔레트 시공까지 완벽한 시각적 청사진

실전 2D 게임 프로젝트 1 (Unity 맵 시스템 기초)

Gemini Notebook

타워 디펜스 코어 게임플레이 루프

1. 웨이브 로직 (Wave)
일정 주기마다 정해진 경로로 적 생성

2. 길 찾기 (Pathfinding)
인공지능(NavMesh 기반)을 통한 적의 이동

3. 공격 메커니즘 (Attack)
적의 이동 경로에 무기를 배치하여 타격

4. 재화 획득 & 업그레이드 (Upgrade)
적 파괴 시 획득한 재화로 무기 강화

5. 스테이지 클리어 (Clear)
목표 지점 방어 성공 시 다음 스테이지로 이동

Gemini Notebook

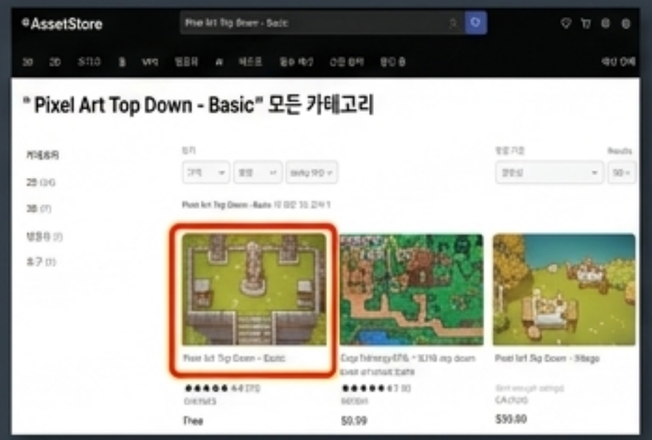
맵 시스템 아키텍처 비교: 통맵 vs 타일 맵

구분	통맵 (Whole Map)	타일 맵 (Tile Map)
제작 방식	거대한 하나의 이미지 사용	작은 타일 조각을 격자에 배치
메모리 효율	낮음 (해상도에 비례해 급증)	매우 높음 (에셋 재사용 극대화)
수정 및 확장	어려움 (이미지 전체 재작업 필요)	쉬움 (팔레트에서 즉각적인 타일 교체 가능)
추천 장르	고정된 화면의 정적인 게임	레벨 디자인 변경이 잦은 RPG, 디펜스

Gemini Notebook

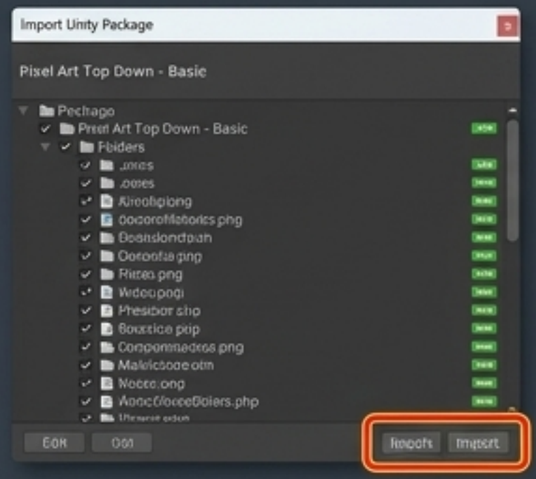
그래픽 리소스 준비 파이프라인

Step 1: 에셋 검색 및 다운로드



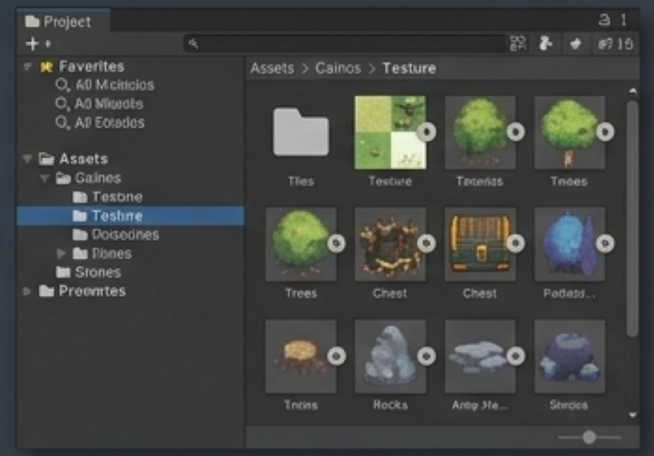
에셋 스토어에서 'Pixel Art Top Down - Basic' 검색 후 [내 에셋에 추가하기].

Step 2: 프로젝트 임포트



[Window]-[My Assets]에서 패키지 임포트.

Step 3: 자재 확인

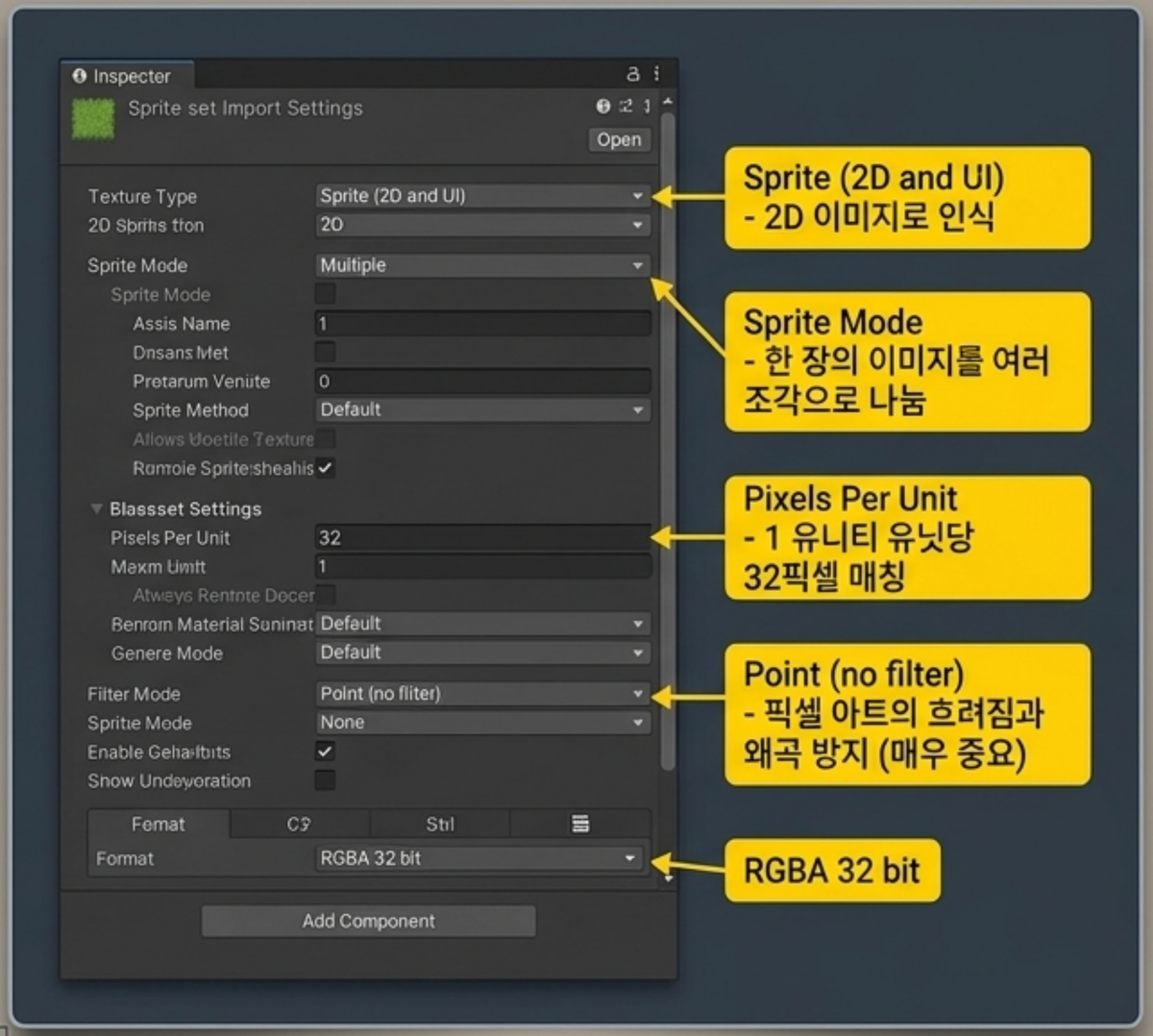


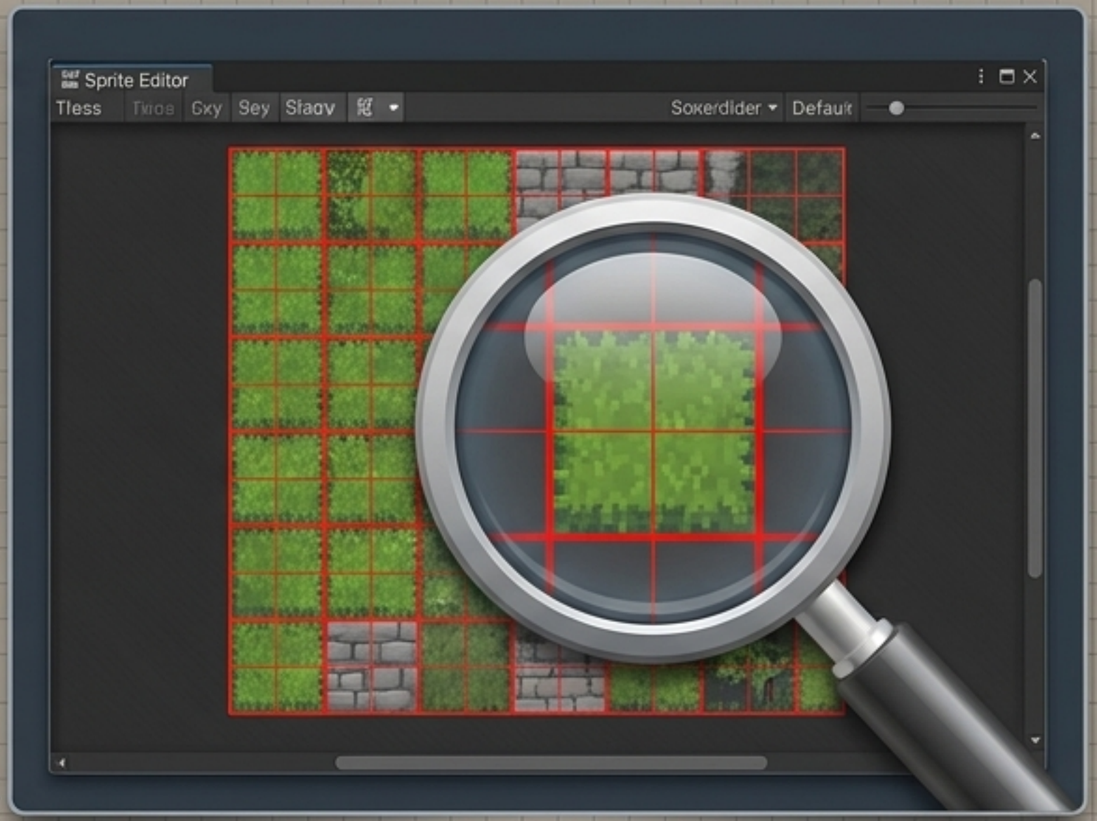
Assets > Cainos > Texture 폴더 내에 타일 및 오브젝트 스프라이트가 성공적으로 적재되었는지 확인.

Gemini Notebook

스프라이트 슬라이싱 해부도 (The Slicing Anatomy)

32 x 32



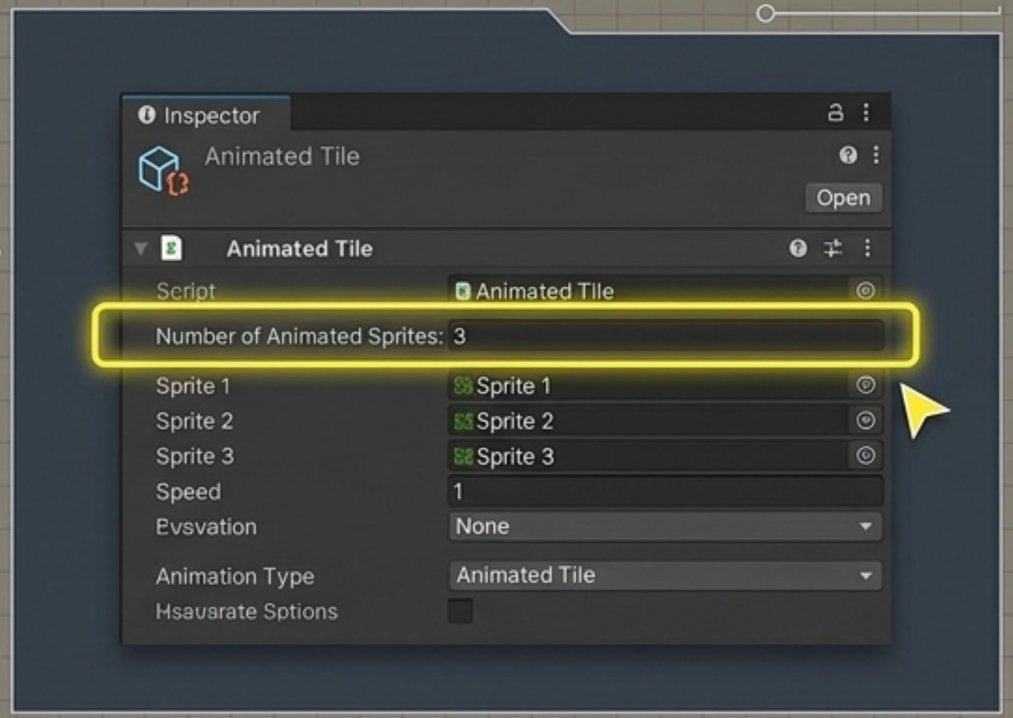


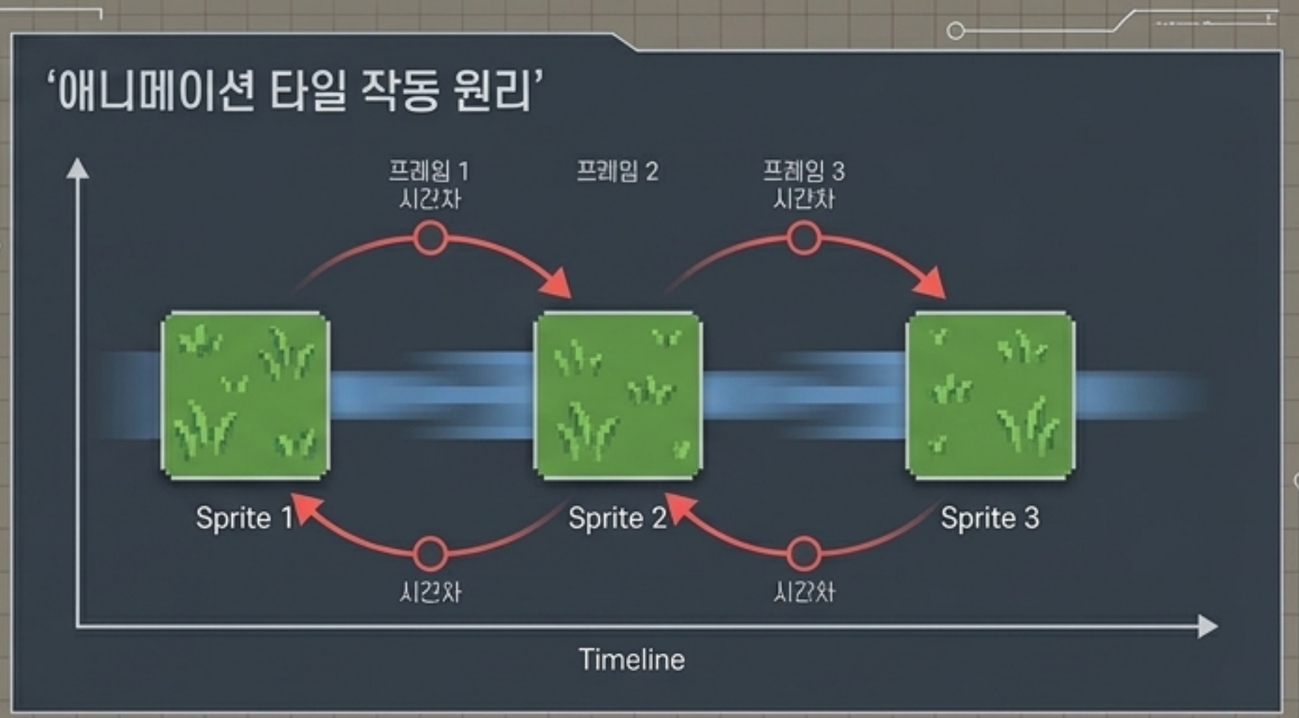
Slice 세팅: Type을 'Grid By Cell Size'로 설정 후
Pixel Size X: 32, Y: 32로 자르기 실행.

Gemini Notebook

타일의 진화: Animated Tile 메커니즘

정적인 이미지 대신, 여러 스프라이트를 시간차를 두고 교체하여 흔들리는 풀잎이나 흐르는 물결을 렌더링합니다.

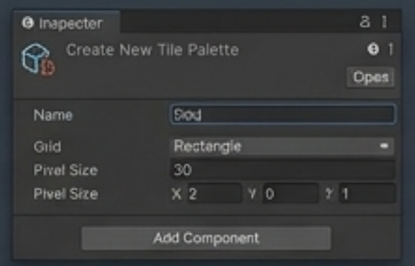




설정 방법: [Create]-[2D]-[Tiles]-[Animated Tile] 생성 후, 사용할 스프라이트 개수(3개)를 지정하고 프레임별 이미지를 드래그 앤 드롭으로 등록합니다. (Speed 속성으로 재생 속도 조절)

Gemini Notebook

디지털 화가의 툴박스: 타일 팔레트 매핑



초기 세팅: [Window]-[2D]-[Tile Palette] 열기
-> Create New Tile Palette (Grid: Rectangle 설정)
-> 생성한(슬라이스한) 타일들을 드래그하여 팔레트에 등록.

Select (화살표):
특정 타일 영역 선택

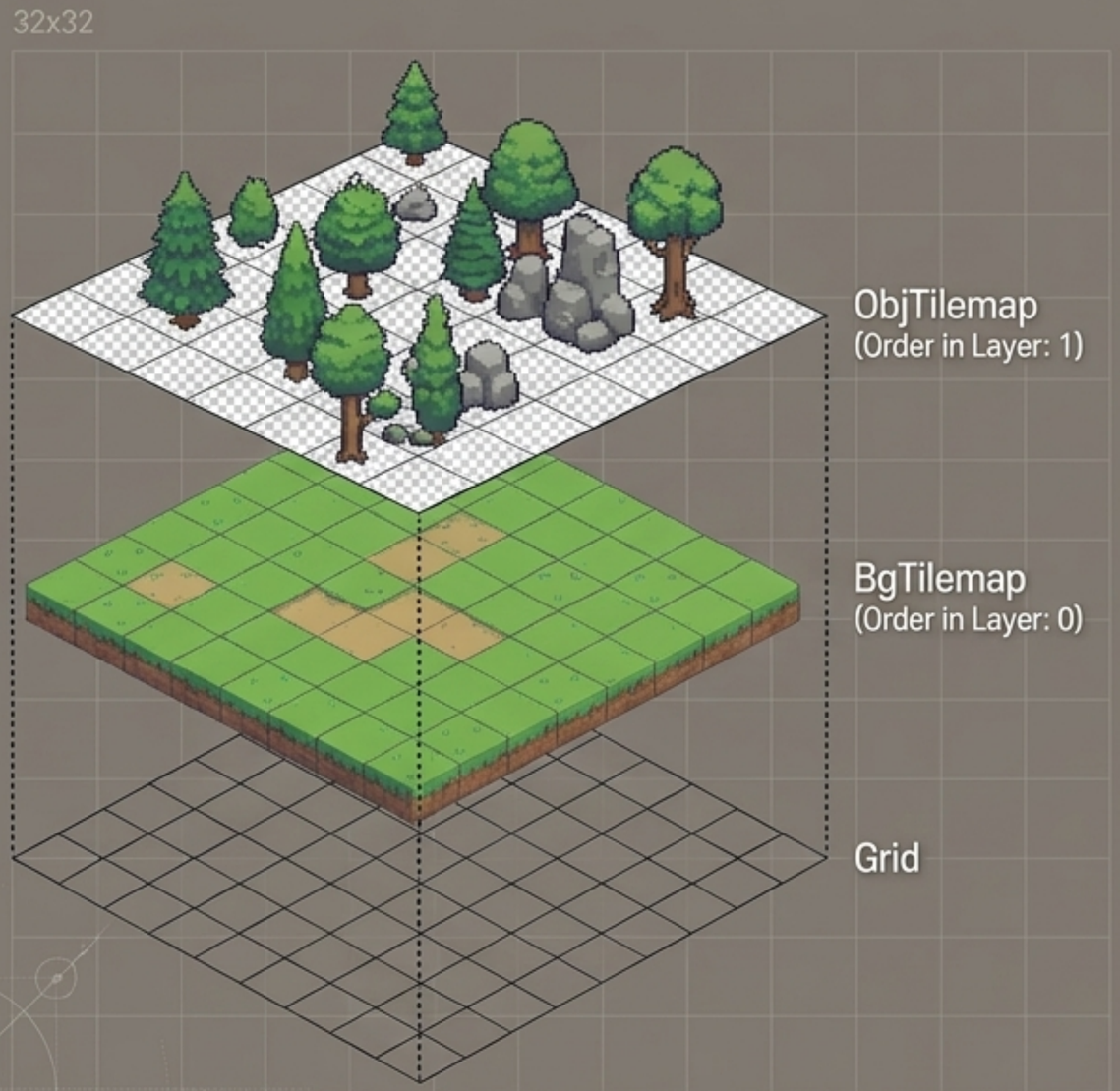
Move (십자):
팔레트 내 타일 배치 이동

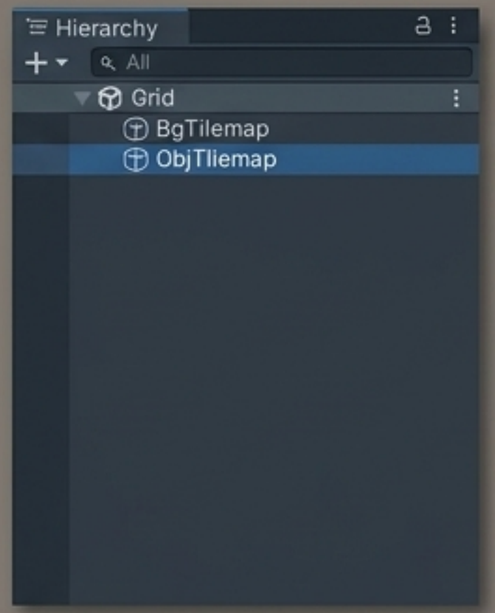
Paint (붓):
씬에 선택한 타일을 그리기

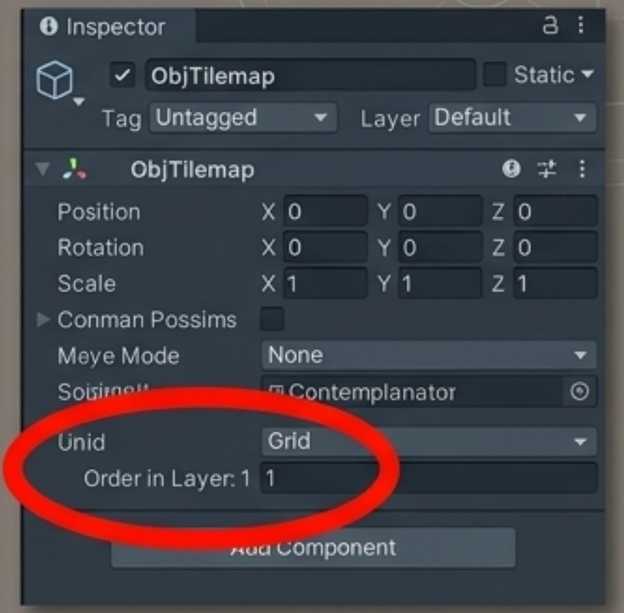
Erase (지우개):
그려진 타일 삭제

Gemini Notebook

레이어드 아키텍처: 배경과 오브젝트의 분리







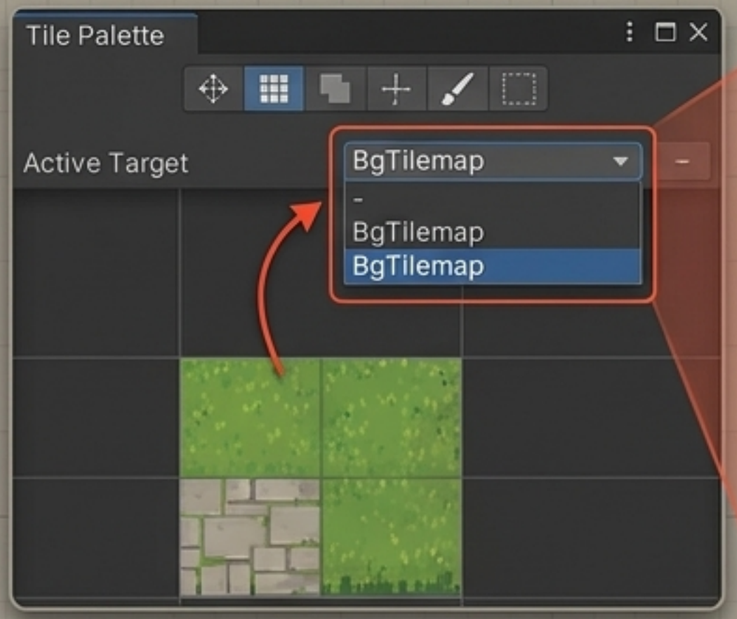
생성 과정: Hierarchy에서 [2D Object]-[Tilemap]-[Rectangular] 선택.
1개의 Grid 아래에 2개의 타일 맵(BgTilemap, ObjTilemap)을 구성.

렌더링 순서 제어:
- BgTilemap: 바닥에 깔리는 지형 (Order in Layer = 0)
- ObjTilemap: 지형 위에 세워지는 장애물/구조물
  (Order in Layer = 1, 숫자가 클수록 앞에 렌더링됨)

Gemini Notebook

맵 드로잉 시공: 캔버스에 생명 불어넣기

Architect's Blueprint



Structural panel

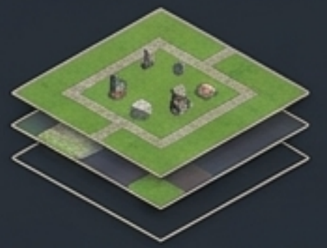
Step 1. 배경 깔기:

타일 팔레트의 Active Target을 `BgTilemap`으로 설정. Paint 툴을 사용해 잔디와 바닥 타일을 채웁니다.

Step 2. 오브젝트 배치:

Active Target을 `ObjTilemap`으로 변경! 그 위에 나무, 들기등, 길 등을 그려 입체감을 완성합니다.

최종 결과:



에셋 가공부터 레이어 설정까지 완벽히 설계된 2D 타워 디펜스의 기초 앱 시스템 구축 완료.

Gemini Notebook

시스템 디자인 블루프린트

2D NavMesh 기반 길 찾기와 웨이브 시스템 구축

Environment [NavMesh]

Actor [Agent]

Director [Wave Manager]

Gemini Notebook

3개의 핵심 모듈이 유기적으로 결합된 시스템 아키텍처

감독 (Director)

웨이브 스폰 매니저
및 난이도 제어

Manager가
Agent를 생성함

Agent 파괴 시
Manager에 보고함

무대 (Environment)

NavMesh 데이터 변환 및
장애물 인지

배우 (Actor)

NPC의 길 찾기 메커니즘과
상태 관리

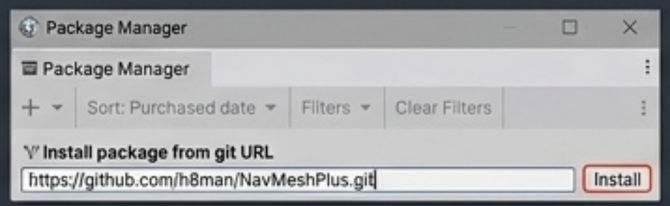
Agent가 NavMesh를
읽고 이동함

Gemini Notebook

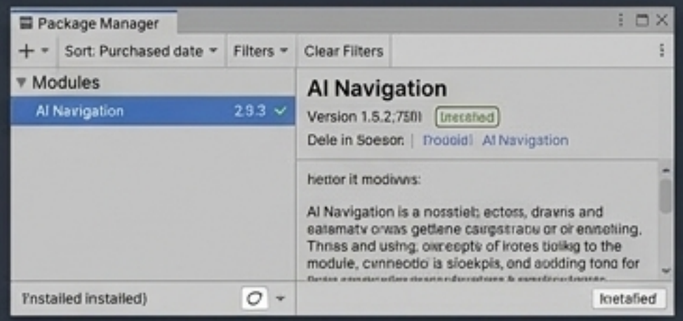
2D 환경에서 NavMesh를 구동하기 위한 의존성 파이프라인

Step 1

Step 2



Step 3

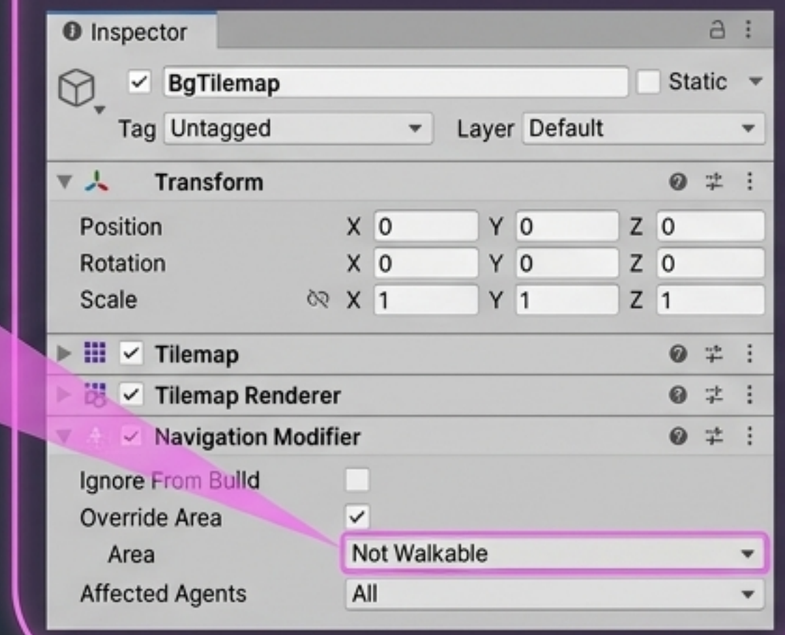


버전 관리 도구(Git) → 커스텀 브리지 패키지(NavMeshPlus) → 공식 인공지능 모듈(AI Navigation)

Gemini Notebook

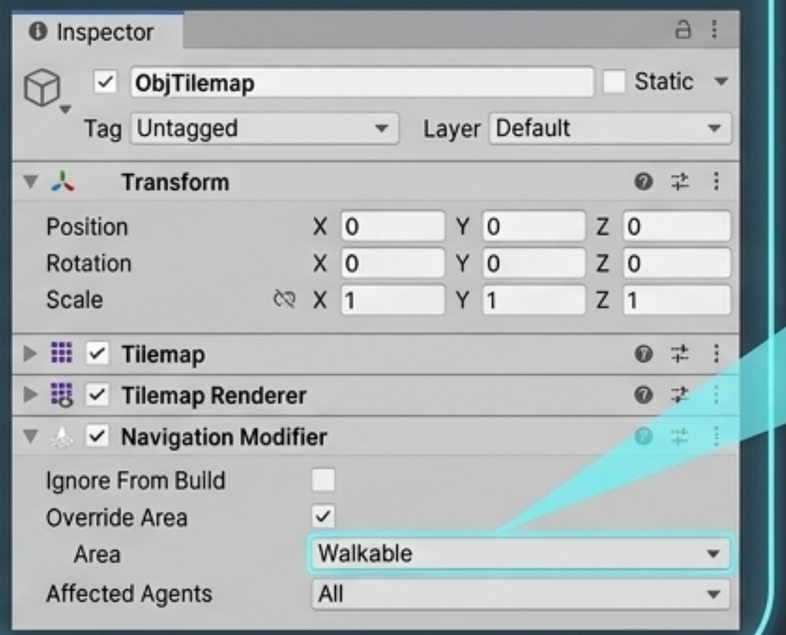
타일맵 속성 분리를 통한 이동 가능 구역과 장애물의 정의

BgTilemap 설정 (장애물)



동일한 컴포넌트,
상반된 Area 속성값 부여

ObjTilemap 설정 (길)



Gemini Notebook

2D 공간을 에이전트가 인식할 수 있는 내비게이션 데이터로 변환

Navigation CollectSources2d (데이터 수집)

NavMeshMgr가 2D 공간의 지형 정보를 스캔합니다.

Rotate Surface to XY (평면 축 회전)

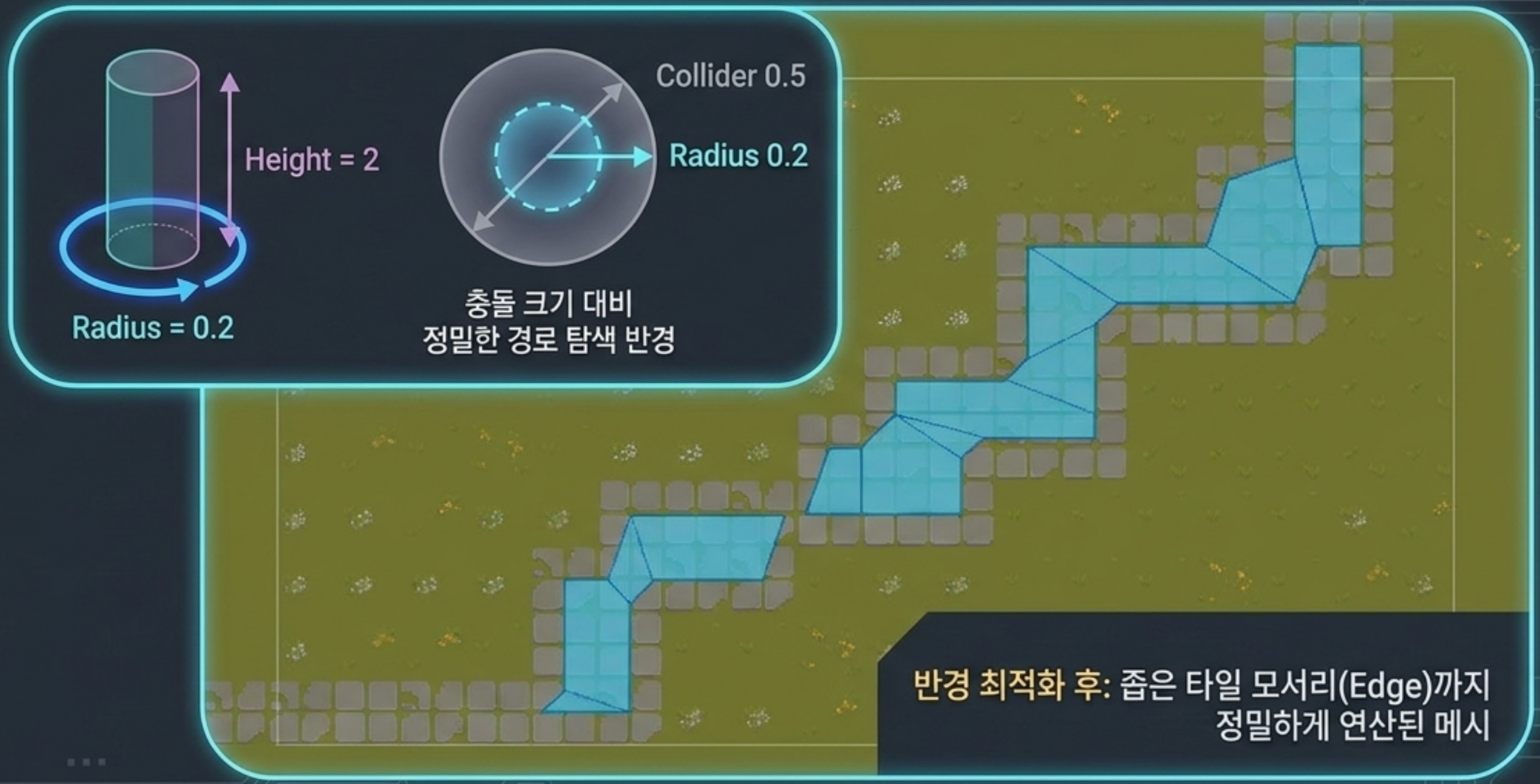
3D 축을 2D XY 평면으로 변환하여 계산을 단순화합니다.

Navigation Surface (연산 및 표면 생성)

최종 내비게이션 메시를 생성(Bake)하여 이동 가능한 영역을 정의합니다.

Gemini Notebook

게임 내 2D 스프라이트 크기에 맞춘 내비게이션 반경 최적화



Gemini Notebook

단일 에이전트의 타겟 추적 프로토타입 구현 및 검증

LOGIC (ForceNavMove.cs)

```
using UnityEngine;
using UnityEngine.AI;

public class ForceNavMove : MonoBehaviour
{
    public Transform endObject;
    private NavMeshAgent enemyAgent;

    void Start()
    {
        enemyAgent = GetComponent<NavMeshAgent>();
        if (endObject != null)
        {
            enemyAgent.SetDestination(endObject.transform.position);
        }
    }
}
```

SIMULATION (A* Pathfinding)

Gemini Notebook

확장성을 고려한 단일 책임 원칙 기반의 NPC 로직 모듈화

Enemy Prefab

Enemy.cs [상태 및 통신]

- 체력(HP) 관리
- 데미지 처리
- 매니저와의 연결

EnemyMove.cs [행동 제어]

- 길 찾기 연산
- 목표 도달 확인
- 자원 회수(Destroy)

Why Modularize?

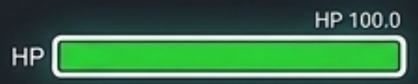
- ☑ 단일 책임 원칙(SRP) 준수
- ☑ 향후 다양한 타입의 적 패턴 추가 용이
- ☑ 유지보수 및 디버깅 효율성 극대화

Gemini Notebook

매니저 시스템과의 통신 채널을 담당하는 상태 관리 로직

Enemy.cs [상태 관리 로직]

```
using UnityEngine;

public class Enemy : MonoBehaviour {

    public float hpValue;
    public EnemyMgr _enemyMgr;

    void Start() {
        hpValue = 100.0f;
    }

    public void SetEnemeyMgr(EnemyMgr enemyMgr) {
        _enemyMgr = enemyMgr;
    }
}
```



양방향 의존성 브리지 개설:
적이 파괴될 때 스폰 매니저에게 '나 죽었어'라고 보고하기 위한 통신 채널 설정

Gemini Notebook

Technical Blueprint / Modern IDE

목표 도달 판단과 자원 회수를 처리하는 이동 제어 메커니즘

로직 흐름 (Logic Flow)

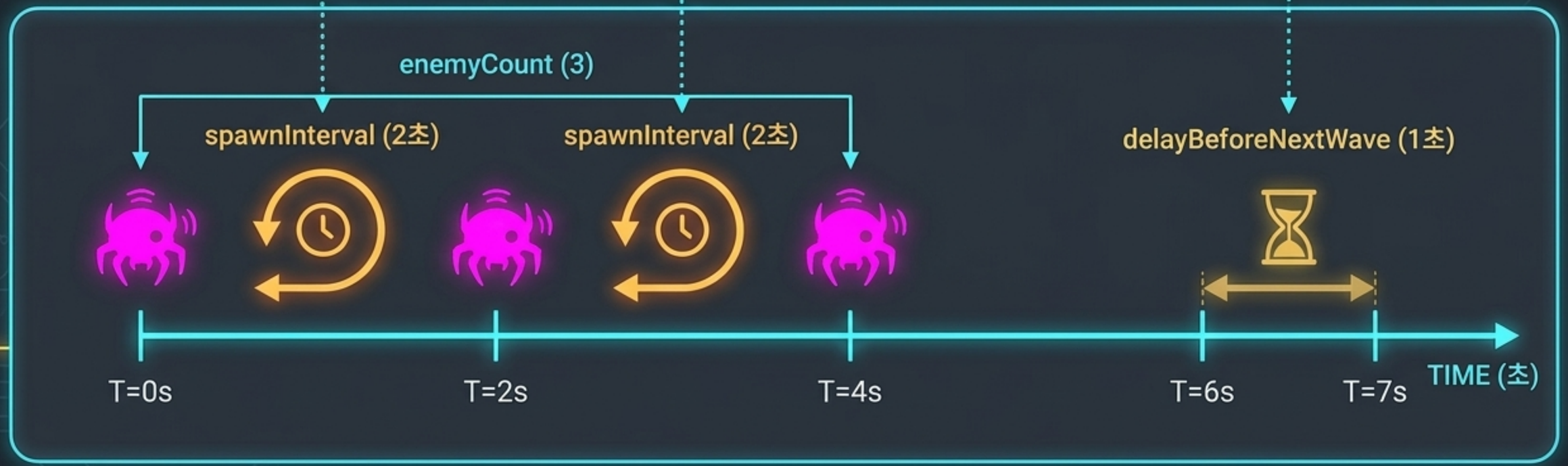
EnemyMove.cs (코드 구현)

```
using UnityEngine;
using UnityEngine.AI;

public class EnemyMove : MonoBehaviour
{
    [Header("이동 설정")]
    public Transform endObject;
    public float arrivalDistance = 0.1f;

    private NavMeshAgent enemyAgent;
    private EnemyMgr enemyMgr;

    private void Start()
    {
        enemyAgent = GetComponent<NavMeshAgent>();
        enemyAgent.updateRotation = false;
        enemyAgent.updateUpAxis = false;

        if (endObject != null)
        {
            enemyAgent.SetDestination(endObject.transform.position);
        }
    }

    public void SetEnemyMgr(EnemyMgr mgr)
    {
        this.enemyMgr = mgr;
    }

    private void Update()
    {
        // 경로 계산 및 목표 도달 확인 로직
        if (!enemyAgent.pathPending && enemyAgent.remainingDistance <= arrivalDistance)
        {
            if (enemyMgr != null)
            {
                enemyMgr.DieEnemyAtDestination();
            }
            Destroy(gameObject);
        }
    }
}
```

Gemini Notebook

데이터 구조화를 통한 적 생성 패턴 및 웨이브 타임라인 설계

```
public class EnemyWaveData
{
    public int enemyCount = 3;
    public float spawnInterval = 2f;
    public float delayBeforeNextWave = 1f;
}
```



Gemini Notebook

무한 루프 메커니즘으로 구동되는 적 스폰 엔진의 작동 원리

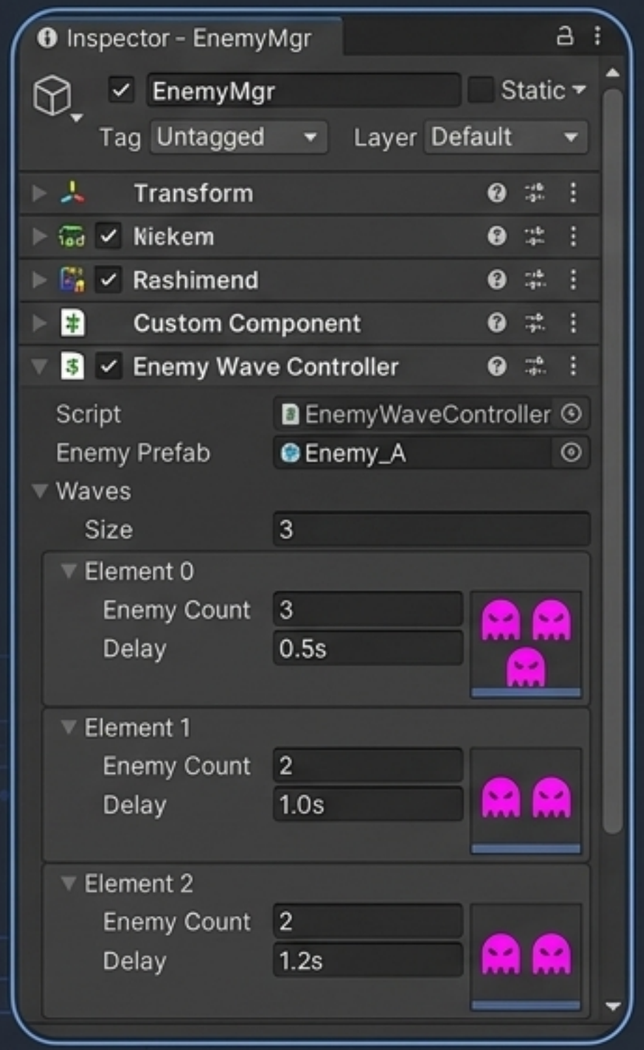
Gemini Notebook

팩토리 패턴을 활용한 객체 인스턴스화 및 의존성 주입

```
// 펙토리 패턴: 객체 생성 및 의존성 주입
void SpawnEnemy()
{
    if (currentWave.enemyCount > spawnedEnemyNumInWave)
    {
        // Stage 1: Instantiate
        GameObject enemy = Instantiate(enemyPrefab,
        spawnPoint.position, spawnPoint.rotation);

        // Stage 2: 의존성 주입 (목표 지점)
        EnemyMove enemyMove = enemy.
           .GetComponent<EnemyMove>();
        enemyMove.SetTarget(endPoint); // 목표 지점 주입

        // Stage 3: 의존성 주입 (상위 매니저 통신)
        Enemy enemyComp = enemy.GetComponent<Enemy>();
        enemyComp.SetEnemyMgr(this); // 싱위 매니저 주입

        spawnedEnemyNumInWave++;
    }
}
```

Gemini Notebook

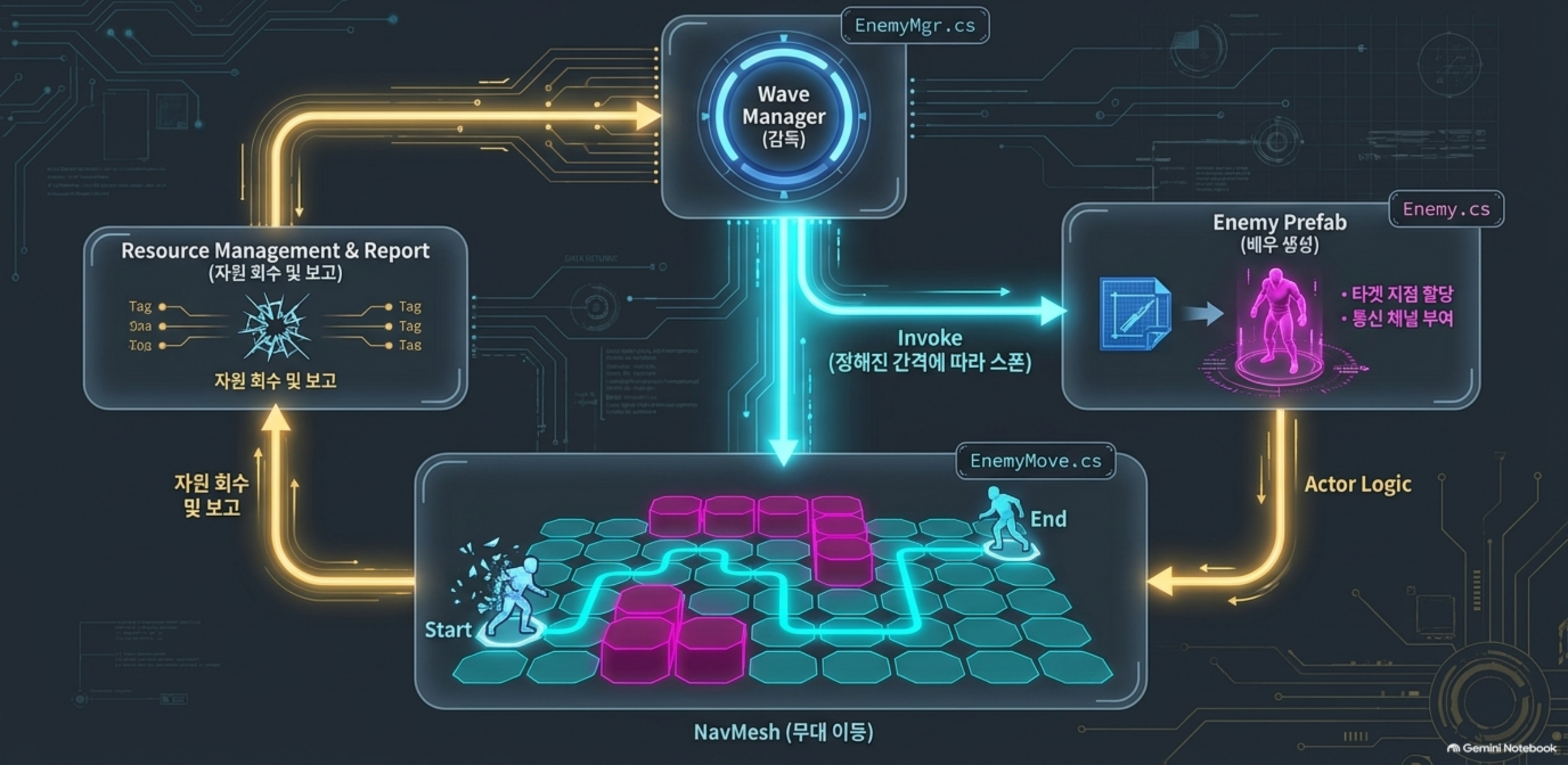
인스펙터 패널을 통한 게임 플레이 난이도 곡선 및 웨이브 제어



코드를 수정하지 않고 기획 데이터(List)만으로
레벨 디자인을 제어하는 Data-driven 아키텍처

Gemini Notebook

무대, 배우, 감독이 하나의 시스템으로 작동하는 마스터 블루프린트



Gemini Notebook

실전 2D 게임 프로젝트: 공격 무기 시스템 아키텍처

타워 생성, 발사체 물리, 그리고 게임 경제 시스템의 완벽한 통합

Gemini Notebook

4개의 핵심 모듈이 만드는 하나의 생태계

무기 생성 (Tower)

적 탐지 및 공격 대기열 관리. 사거리 내 가장 가까운 적을 추적하는 핵심 로직.

발사체 (Projectile)

물리 엔진 기반 이동 및 타격 판정. 적중 시 데미지 전달 후 소멸.

타워 건설 (TowerMgr)

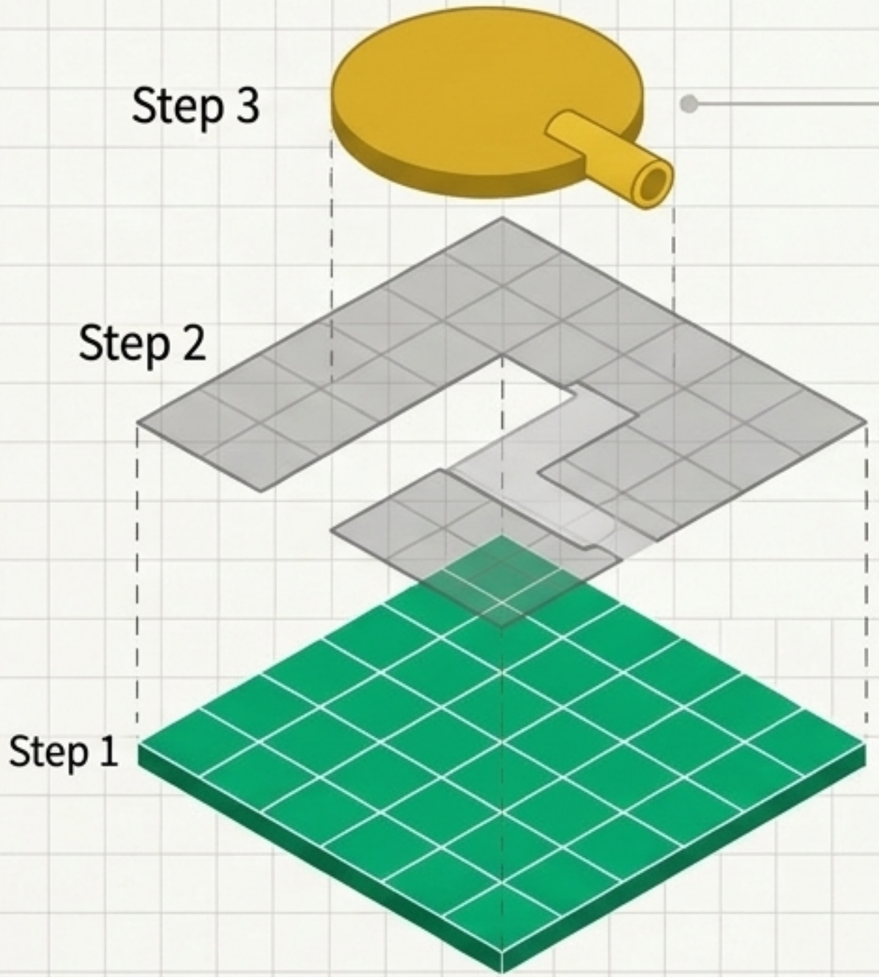
마우스 입력 좌표 변환 및 그리드(Grid) 중복 건설 방지. 실질적인 생성자.

게임 상태 (GameMgr)

재화(Gold) 검증 및 기지 체력(HP) 통합 관리. 시스템의 최고 관리자.

Gemini Notebook

타워 오브젝트 설계: 공간과 렌더링



TowerObj (Sprite)

Circle Sprite, Color Yellow, Order in Layer 3.
공격을 수행할 시각적 본체.

Navigation Modifier

Area = Not Walkable.
NavMeshMgr에서 Bake하여 적의 이동 경로를
물리적으로 방해하는 길찾기 데이터.

Tilemap (Building)

Order in Layer 1, Color Green.
배치 가능 구역을 시각화하는
베이스 레이어.

핵심 인사이트:
시각적 렌더링(Order in Layer)과
논리적 경로(NavMesh)를 독립된
컴포넌트로 결합하여 시스템의
유연성을 확보합니다.

Gemini Notebook

가장 가까운 적을 추적하는 스마트 타워의 뇌

```
using UnityEngine;

public class Tower : MonoBehaviour
{
    [Header("타워의 속성")]
    public GameObject projectTile; // 발사체
    public float attackRange = 100.0f;
    public float waitAttackTime = 3.0f;

    private EnemyMgr enemyMgr;
    private GameObject findEnemy = null;
    private float curAttackTime = 0.0f;

    void FixedUpdate()
    {
        curAttackTime += Time.fixedDeltaTime;

        // 사정거리 인의 작을 탐색하여 가장 가까운 작을 확인
        if (curAttackTime > waitAttackTime)
        {
            if (this.enemyMgr)
            {
                findEnemy = this.enemyMgr.GetClosestEnemy(
                    transform.position,
                    attackRange
                );

                if (findEnemy)
                {
                    // 발사체 생성 및 데이터 전달
                    GameObject prajectile = Instantiate(projectTile, transform.position, Quaternion.identity);
                    projectile.GetComponent<Projectile>().SetTarget(findEnemy);
                    projectile.GetComponent<Projectile>().SetDirection(transform.up);
                    // (Manager info passing omitted for brevity)
                }
                curAttackTime = 0.0f;
            }
        }
}
```

1. 쿨다운 대기

curAttackTime > waitAttackTime (공격 속도 제어)

2. 표적 탐색

EnemyMgr 호출. 사정거리(attackRange) 내 최단 거리 적 식별

3. 발사체 생성

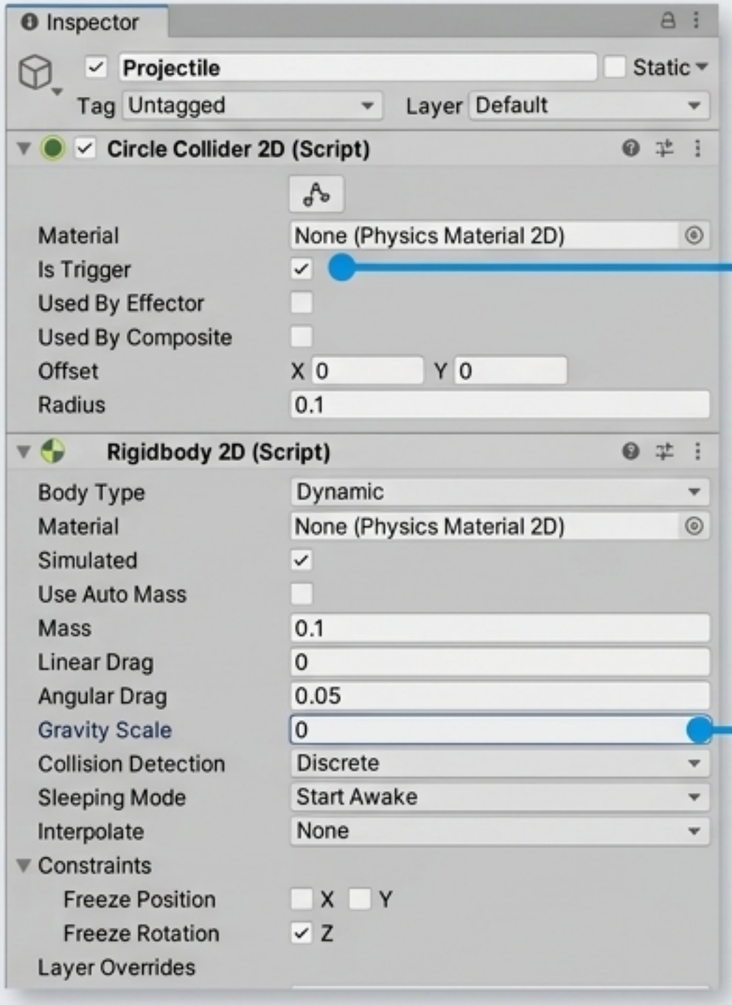
프리팹 Instantiate 호출

4. 데이터 전달

생성된 발사체에 표적(Target), 방향(Direction),
관리자(Manager) 정보 주입

Gemini Notebook

발사체 물리 엔진 설정: 충돌과 통과



Circle Collider 2D

Is Trigger = True 활성화 필수.
물리적 튕김 현상(Bounce) 없이 오직 데미지 판정을 위한 오버랩 이벤트만 발생시킵니다.

Rigidbody 2D

Gravity Scale = 0.
2D 탑다운 환경에서 중력의 영향을 배제하여 발사체가 설정된 직선 궤도로만 날아가도록 강제합니다.

Gemini Notebook

목표를 향한 비행과 타격 판정

```
void OnTriggerEnter2D(Collider2D collision)
{
    if(collision.CompareTag("Enemy")) // 3. Validation
    {
        Enemy enemy = collision.GetComponent<Enemy>();
        if(enemy != null)
        {
            enemy.OnDamage(attackVal); // 4. Impact
        }
        Destroy(gameObject); // 4. Impact
    }
}
```

1. 이동 (Flight)

FixedUpdate를 통한 등속 이동.
(수명: lifeTime = 5f)

2. 충돌 감지 (Detection)

OnTriggerEnter2D 이벤트 발동.

3. 피아 식별 (Validation)

CompareTag("Enemy")를 통해 대상이 적군인지 엄격히 확인.

4. 타격 및 소멸 (Impact)

enemy.OnDamage(attackVal) 호출로 데미지 적용 후 Destroy(gameObject)로 즉시 메모리 해제.

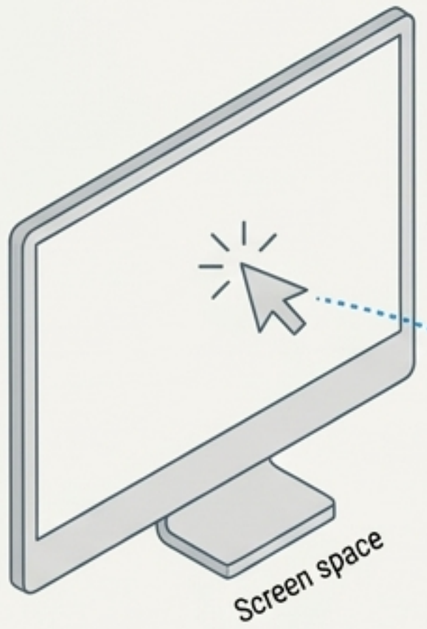
Gemini Notebook

아키텍처 비교: 타워 vs 발사체

	타워 (Tower)	발사체 (Projectile)
역할 (Role)	탐지 및 생성자 (Spawner)	전달 및 파괴자 (Effector)
수명 (Lifespan)	영구적 (파괴 전까지)	일시적 (lifeTime 도달 또는 충돌 시 즉각 해제)
이동 (Movement)	고정 (정적)	동적 (Rigidbody 2D 방향 이동)
물리 처리 (Physics)	NavMesh 경로 방해	Trigger 전용 (통과 및 이벤트 발생)
의존성 (Dependency)	TowerMgr 및 EnemyMgr에 의존	발사한 Tower가 주입한 데이터에 의존

Gemini Notebook

마우스 클릭에서 타워 생성까지



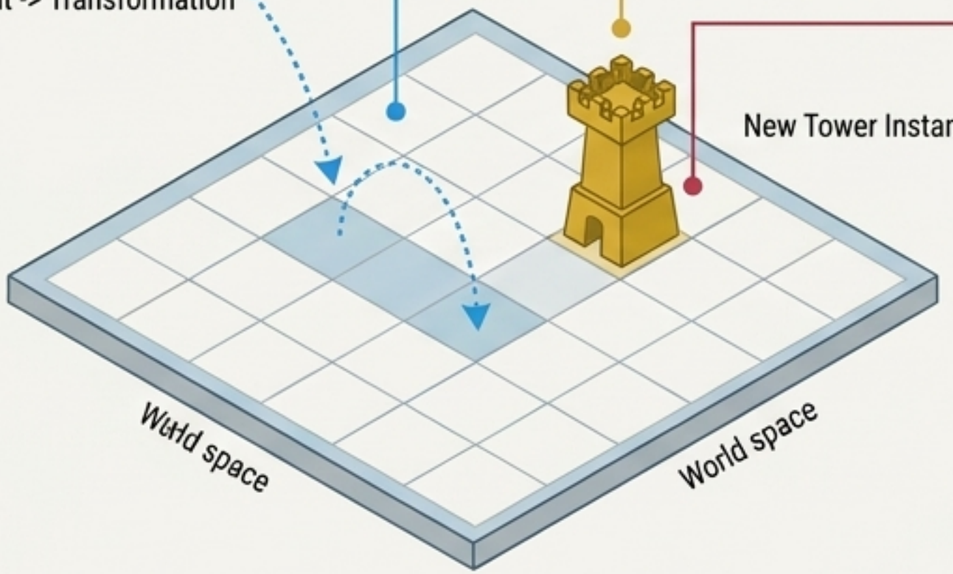
Screen space

Input -> Transformation

Step 1: 좌표 변환

```
Vector2 mousePos = Mouse.current.position.ReadValue();
Vector3 mouseWorldPos = mainCamera.ScreenToWorldPoint(
    new Vector3(mousePos.x, mousePos.y, mainCamera.nearClipPlane)
);
Vector3Int cellPosition = buildTileMap.WorldToCell(mouseWorldPos);
```

ScreenToWorldPoint -> WorldToCell. 화면의 픽셀 좌표를 타일맵의 논리적 그리드(Grid) 셀 좌표로 정밀하게 변환합니다.



World space

New Tower Instance

Step 2: 중복 방지 시스템

```
private Dictionary<Vector3Int, GameObject> CanBuildTower =
    new Dictionary<Vector3Int, GameObject>();
...
if (CanBuildTower.ContainsKey(cellPosition))
{
    // 해당 위치에 타워이 있는지 확인
    return; // 이미 타워가 존재하면 건설 중단
}
```

Dictionary<Vector3Int, GameObject>를 활용. 이미 건설된 셀 좌표의 키(Key) 값을 검사하여 동일 위치 중복 건설을 원천 차단합니다.

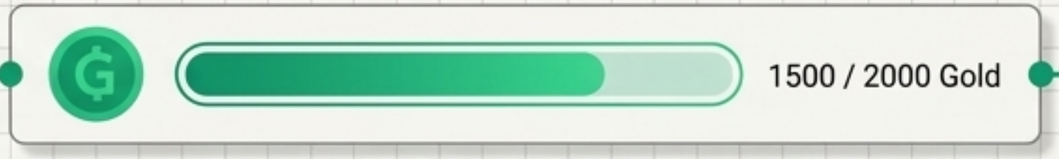
Step 3: 재화 검증 및 생성

```
// gameMgr.checkGold(towerPrice) 통과 시
GameObject newTower = Instantiate(buildTower, ...);
newTower.transform.SetParent(transform);
CanBuildTower.Add(cellPosition, newTower);
```

checkGold() 통과 시 Instantiate 실행. 생성된 타워는 계층 구조 정리를 위해 TowerMgr의 자식 노드로 SetParent 됩니다.

Gemini Notebook

게임의 맥박: 재화와 체력 관리




```
// Gold Management
public bool checkGold(int cost) {
   return curGold >= cost;
}

public void consumeGold(int cost) {
   if (checkGold(cost)) {
      curGold -= cost;
      UpdateGoldUI();
   }
}
```

Gold System

- curGold: 타워 건설의 연료.
- checkGold(): 구매 가능 여부 반환 (Booleans).
- consumeGold(): 건설 시 재화 차감.

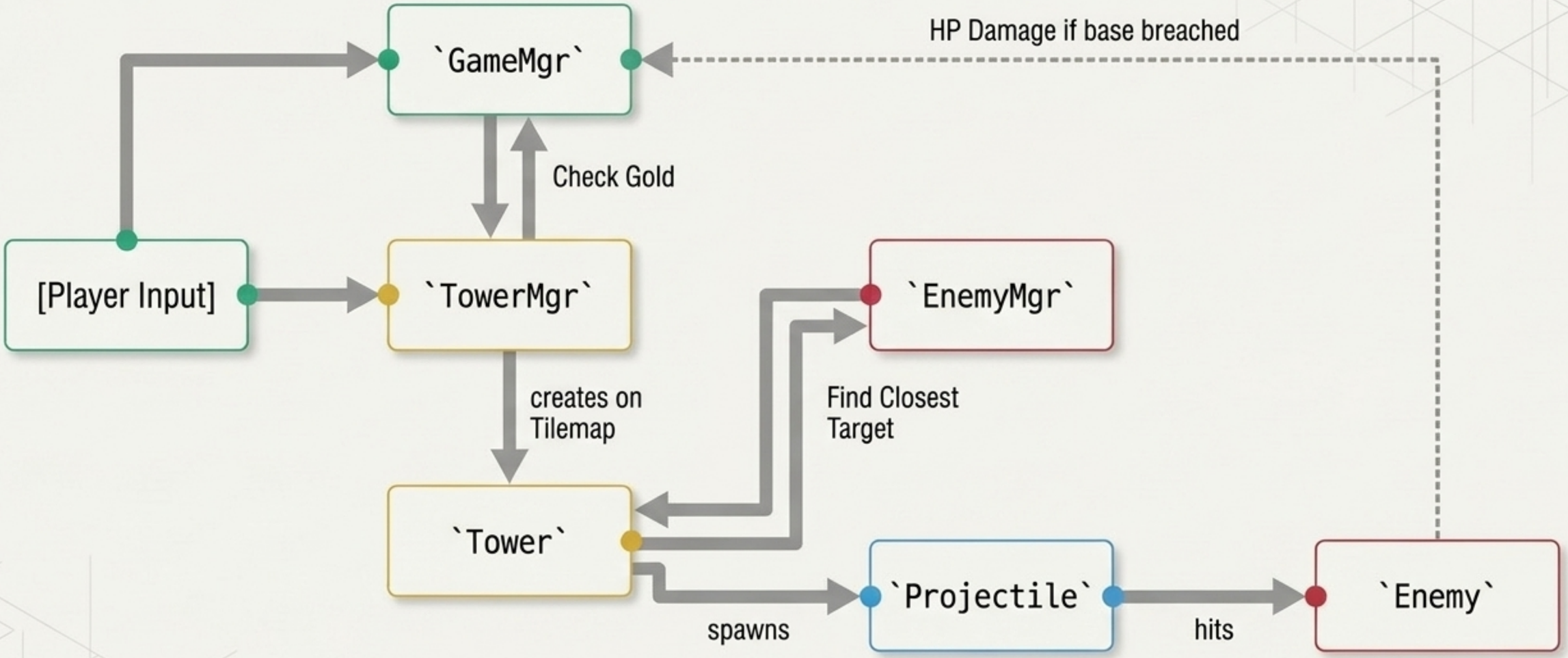
```
// HP Management
public void damageHP(int damage) {
   curHP -= damage;
   UpdateHPUI();
   if (curHP <= 0) {
      GameOver();
   }
}

private void GameOver() {
   Time.timeScale = 0f; // Pause game
   ShowGameOverScreen();
}
```

HP System

- curHP: 기지의 생명력.
- damageHP(): 적이 기지에 도달 시 체력 감소.
- GameOver(): curHP <= 0 달성 시 게임 루프 종료.

Gemini Notebook

공격 시스템 종합 아키텍처



Gemini Notebook

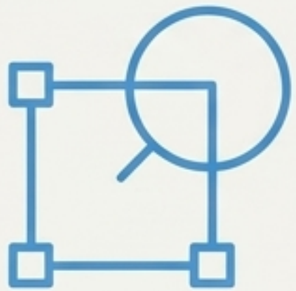
안정적인 게임 생태계 구축을 위한 3대 원칙

1. 완벽한 모듈화 (Modularity)

Tower는 탐지만, Projectile은 타격만 담당합니다. 책임을 명확히 분리하여 코드의 유지보수성과 확장성을 극대화했습니다.

2. 중앙 제어 (Centralized Control)

개별 유닛이 자원을 직접 관리하지 않고 GameMgr와 TowerMgr라는 매니저 클래스를 통해 상태를 일원화하여 치명적인 데이터 충돌 버그를 차단합니다.



3. 물리 연산 최적화 (Physics Optimization)

무거운 물리적 충돌(Bounce) 연산 대신 Is Trigger를 활용한 가벼운 이벤트 감지와 NavMesh 길찾기 시스템의 결합으로 모바일 환경에서도 안정적인 퍼포먼스를 보장합니다.

Gemini Notebook

시스템 엔진 구현 완료.
이제 게임 플레이가 시작됩니다.

건설 시스템, 물리 엔진, 재화 관리가 하나의 유기적인 생태계로 완성되었습니다.

다음 단계: 설계된 무기 시스템을 적 웨이브 생성기(Enemy Wave System)와 결합하여 당신만의 완벽한 디펜스 게임을 완성해 보세요.

Gemini Notebook